HOSTING DE UN SERVIDOR DE MINECRAFT FORGE EN UN ENTORNO VIRTUALIZADO

18 DE ENERO DE 2026

Índice

1. Objetivos y alcance del proyecto 3
2. Tecnologías utilizadas 3
2.1. Hardware 3
2.2. Sistema Operativo 3
2.3. VirtualBox 3
2.4. CasaOS 3
2.5. Crafty Controller 4
2.6. Playit.gg 4
2.7. Minecraft Forge y mods 4
3. Arquitectura del sistema 4
3.1. Visión general 4
3.2. Diagrama de arquitectura 4
3.3. Flujo de conexión de jugadores 5
4. Instalación del entorno base 5
4.1. Creación de la máquina virtual 5
4.2. Instalación de Debian 12 (sin GUI) 6
4.3. Configuración de red en VirtualBox 7
5. Configuración de CasaOS 8
5.1. Instalación 8
5.2. Acceso desde el host 9
5.3. Servicios gestionados 9
6. Gestión del servidor con Crafty 9
6.1. Instalación de Crafty 9
6.2. Creación del servidor Forge 10
6.3. Gestión de memoria y rendimiento 11
6.4. Inicio automático y detección de crasheos 11
6.5. Backups automáticos 12
7. Servidor Minecraft Forge con mods 12
7.1. Preparación del servidor 12
7.2. Gestión de mods (cliente y servidor) 13
7.3. Uso de Modrinth para gestionar mods en el cliente 13
7.4. Generación y reinicio del mundo 13
7.5. Permisos y administración 14
8. Acceso remoto con Playit.gg 14
8.1. Problemas con Port Forwarding tradicional 14
8.2. Instalación del agente Playit 14

NELLY PIÑEIRO FERNÁNDEZ

8.3. Creación del tunel 15
8.4. Pruebas de conexión externas 15
9. Pruebas y validación del sistema 15
10. Problemas comunes y soluciones 16
11. Conclusiones y mejoras futuras 17
11.1. Conclusiones 17
11.2. Mejoras futuras 17

NELLY PIÑEIRO FERNÁNDEZ

1. Objetivos y alcance del proyecto

Este proyecto consiste en el diseño e implementación de un servidor Minecraft Forge con soporte para mods, accesible desde Internet, desplegado en un entorno Linux sin interfaz gráfica. Se plantea como un ejercicio práctico para afianzar conocimientos en administración de sistemas y redes, incluyendo virtualización con VirtualBox, gestión de servicios con CasaOS y administración de servidores de juegos mediante Crafty Controller.

El servidor está orientado a un uso reducido (hasta dos jugadores), priorizando la estabilidad, la compatibilidad de mods y la facilidad de mantenimiento. Para facilitar el acceso externo desde redes domésticas con NAT y firewalls restrictivos, se utiliza Playit.gg como solución de túnel.

El proyecto abarca desde la creación y configuración de la máquina virtual hasta la validación del acceso local y remoto, incluyendo la migración del sistema entre equipos. Quedan fuera del alcance aspectos como el desarrollo de mods, la optimización para grandes servidores o el uso de infraestructuras cloud.

2. Tecnologías utilizadas

Para la realización de este proyecto se han empleado distintas tecnologías de hardware y software, seleccionadas con el objetivo de simular un entorno realista de homelab, priorizando la estabilidad, la simplicidad de mantenimiento y el aprendizaje práctico.

2.1. Hardware

El entorno se ha desplegado inicialmente en un PC de escritorio y posteriormente migrado a un portátil. Ambos cuentan con 16 GB de memoria RAM permitiendo asignar recursos adecuados tanto a la máquina virtual como al servidor Minecraft, manteniendo un margen suficiente para el sistema anfitrión.

2.2. Sistema Operativo

Como sistema operativo se ha utilizado **Debian 12** sin entorno gráfico. Esta elección permite reducir el consumo de recursos, mejorar la estabilidad y trabajar directamente con la terminal, acercándose a entornos reales de servidores Linux.

2.3. VirtualBox

La virtualización se ha realizado mediante **Oracle VirtualBox**, utilizado para crear, ejecutar y gestionar la máquina virtual que aloja todos los servicios del proyecto. VirtualBox ha permitido, además, exportar e importar la máquina virtual entre distintos equipos físicos, facilitando la migración del entorno sin necesidad de reinstalaciones.

2.4. CasaOS

CasaOS se ha empleado como plataforma de gestión de servicios autoalojados. Proporciona una interfaz web accesible desde el equipo anfitrión, desde la cual se pueden gestionar

NELLY PINEIRO FERNÁNDEZ

aplicaciones y servicios de forma centralizada, simplificando la administración del sistema pese a no disponer de interfaz gráfica en el sistema operativo virtualizado.

2.5. Crafty Controller

La administración del servidor de Minecraft se ha realizado mediante **Crafty Controller**, una herramienta web que permite crear, configurar y gestionar servidores de Minecraft de forma sencilla. Crafty facilita la gestión de versiones, la asignación de memoria, el control de logs, el inicio automático del servidor y la detección y recuperación ante crasheos.

2.6. Playit.gg

Para permitir el acceso remoto al servidor desde redes externas se ha utilizado **Playit.gg**, una solución de túneles que evita la necesidad de configurar port forwarding y reglas de firewall en el router doméstico. Esta herramienta resulta especialmente útil en entornos con NAT restrictivo o donde no se dispone de control total sobre la red.

2.7. Minecraft Forge y mods

El servidor se basa en **Minecraft Forge**, permitiendo la carga de mods que amplían la jugabilidad. Tanto el servidor como los clientes usan la misma versión de Forge y los mismos mods, garantizando compatibilidad. Para gestionar los mods en el cliente se utiliza Modrinth, que facilita crear perfiles Forge y añadir los mods necesarios antes de conectarse.

3. Arquitectura del sistema

La arquitectura del sistema se ha diseñado con el objetivo de mantener una estructura clara, modular y fácil de mantener, separando correctamente el entorno físico, la virtualización, la gestión de servicios y el acceso de los clientes al servidor de juego.

3.1. Visión general

El sistema se basa en un equipo físico que actúa como host y ejecuta una máquina virtual mediante VirtualBox. Dentro de esta máquina virtual se ejecuta Debian 12 sin entorno gráfico, sobre el cual se instalan y gestionan los distintos servicios necesarios para el funcionamiento del servidor Minecraft.

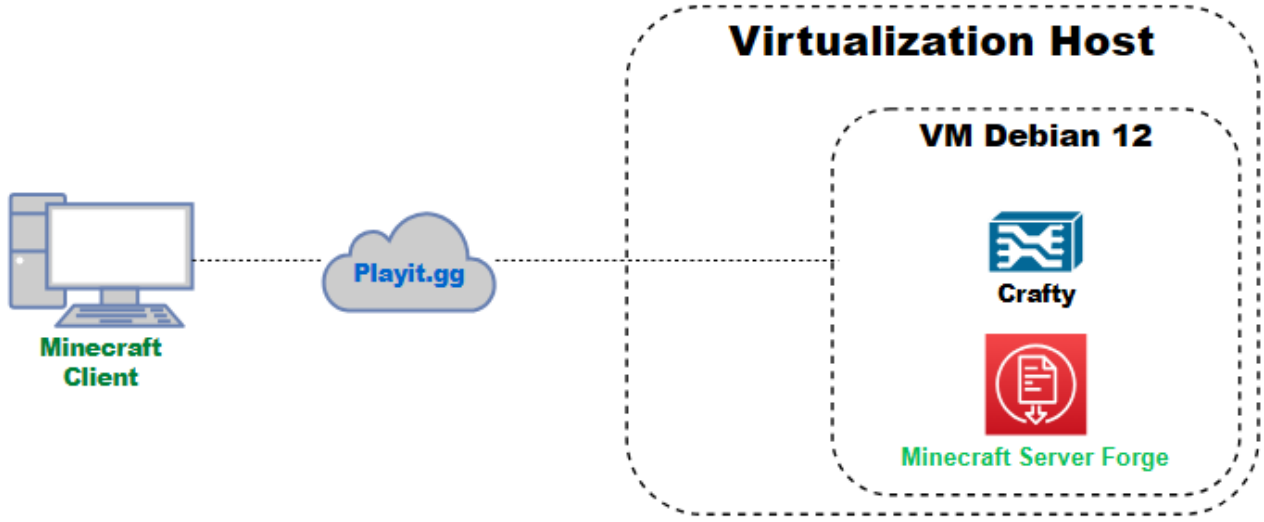
CasaOS proporciona una capa de gestión accesible mediante interfaz web, desde la cual se administra Crafty Controller. Crafty, a su vez, es el encargado de gestionar el ciclo de vida del servidor Minecraft Forge, incluyendo su arranque, parada, configuración y supervisión.

Para permitir el acceso desde Internet, se emplea Playit.gg, que establece un túnel saliente desde la máquina virtual hacia la infraestructura de Playit, exponiendo el servidor mediante una dirección pública accesible por los clientes, sin necesidad de abrir puertos en el router.

3.2. Diagrama de arquitectura

A nivel conceptual, la arquitectura del sistema puede representarse de la siguiente forma:

NELLY PIÑEIRO FERNÁNDEZ



El host físico únicamente actúa como plataforma de virtualización, mientras que toda la lógica del servidor se encuentra encapsulada dentro de la máquina virtual.

3.3. Flujo de conexión de jugadores

El flujo de conexión comienza cuando un jugador se conecta desde su cliente de Minecraft, utilizando la dirección pública proporcionada por Playit.gg. La conexión se enruta a través de la red de Playit hasta el agente instalado en la máquina virtual. Desde ahí, el tráfico se redirige internamente al puerto del servidor Minecraft gestionado por Crafty.

Este enfoque elimina la necesidad de configuraciones complejas de red y reduce los problemas habituales relacionados con NAT, firewalls y direcciones IP dinámicas, proporcionando una solución estable y reutilizable en distintos entornos físicos.

4. Instalación del entorno base

4.1. Creación de la máquina virtual

Para el desarrollo de este proyecto se creó una máquina virtual dedicada que actúa como servidor del entorno Minecraft, permitiendo aislar el sistema de juego del sistema operativo anfitrión y facilitando su mantenimiento y posterior migración a otros equipos.

Antes de crear la máquina virtual, se descargó una **imagen ISO de Debian 12 desde la web oficial del proyecto Debian**, garantizando el uso de un sistema operativo estable, seguro y mantenido a largo plazo.

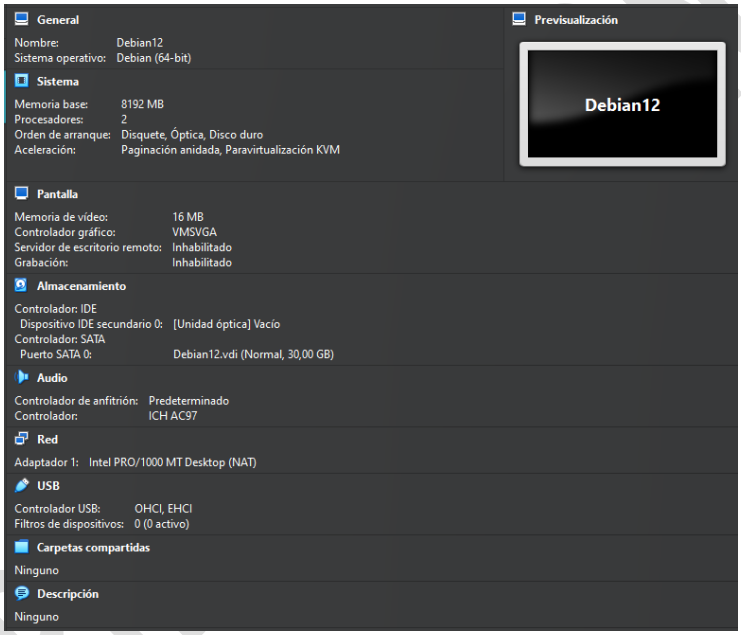
NELLY PIÑEIRO FERNÁNDEZ

Índice de versiones

Consulte las Preguntas frecuentes de Debian si desea una explicación de dónde surgieron estos nombres en clave.

Versión	Nombre clave	Fecha de publicación	Fin del ciclo de vida («end of life», EOL)	Fin del LTS	Fin del ELTS	Estado
15	Duke	Por determinar	Por determinar	Por determinar	Por determinar	Nombre clave anunciado
14	Forky	Por determinar	Por determinar	Por determinar	Por determinar	«testing» — sin fecha de publicación establecida
13	Trixie	09-08-2025	09-08-2028	30-06-2030	30-06-2035	Versión «estable» actual
12	Bookworm	10-06-2023	10-06-2026	30-06-2028	30-06-2033	Versión «oldstable» actual
11	Bullseye	14-08-2021	14-08-2024	31-08-2026	30-06-2031	Versión «oldoldstable» actual, bajo soporte LTS
10	Buster	06-07-2019	10-09-2022	30-06-2024	30-06-2029	Versión archivada, bajo soporte LTS extendido pagado por terceros
9	Stretch	17-06-2017	18-07-2020	01-07-2022	30-06-2027	Versión archivada, bajo soporte LTS extendido pagado por terceros

La máquina virtual se creó utilizando **VirtualBox**, configurando un sistema Linux de tipo Debian y asignando recursos acordes al uso previsto del servidor. Se asignaron **2 núcleos de CPU** y **8 GB de memoria RAM**, suficientes para ejecutar el sistema base y un servidor Minecraft con Forge y múltiples mods de forma estable.

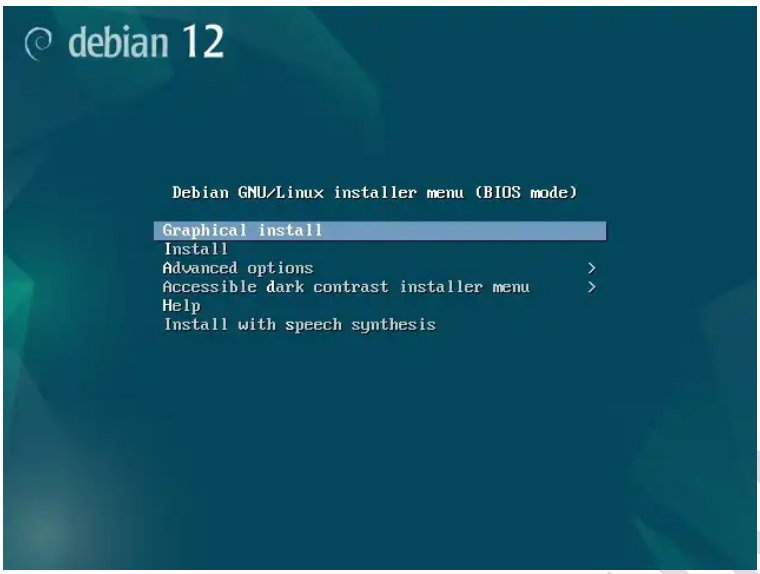


El almacenamiento se configuró mediante un disco virtual en formato **VDI**, con asignación dinámica de 30 GB, suficiente para el sistema operativo, CasaOS, Crafty Controller y los datos persistentes del servidor Minecraft, como mundos, configuraciones y mods.

La máquina virtual se creó **sin entorno gráfico**, ya que toda la administración del sistema se realiza mediante acceso remoto y paneles web, reduciendo el consumo de recursos y optimizando el rendimiento del servidor.

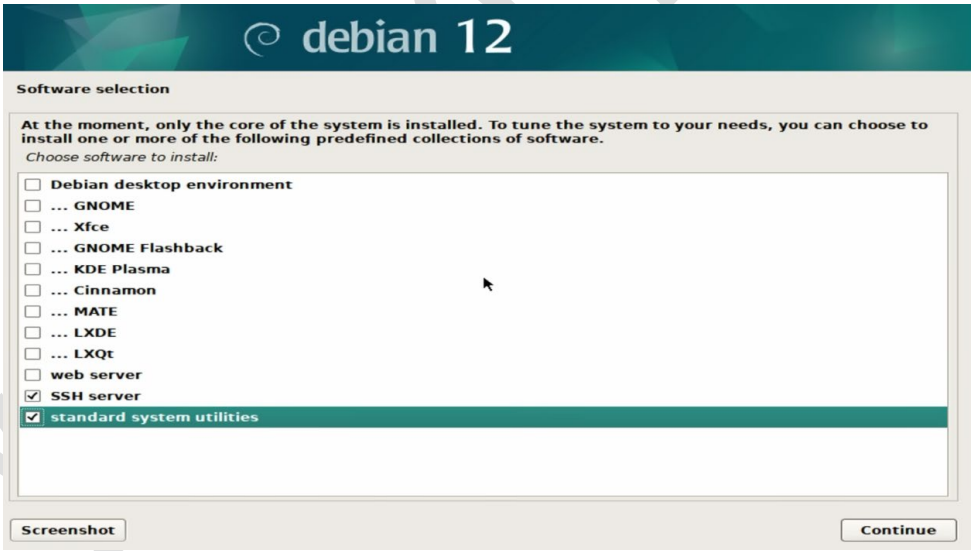
4.2. Instalación de Debian 12 (sin GUI)

La instalación del sistema operativo se realizó utilizando **Debian 12**, a partir de la imagen ISO descargada desde la web oficial del proyecto Debian.



Durante el proceso se configuraron los parámetros básicos del sistema, como idioma, zona horaria y usuario administrador, asegurando el acceso mediante `sudo` para tareas de administración.

En la selección de software se optó por una instalación **mínima**, desmarcando cualquier entorno de escritorio y dejando únicamente los componentes esenciales del sistema, con el objetivo de reducir el consumo de recursos y mejorar la estabilidad del servidor.



Una vez finalizada la instalación y tras el primer reinicio, el sistema quedó listo para su administración por terminal y para la instalación de los servicios necesarios en los siguientes apartados.

4.3. Configuración de red en VirtualBox

La red de la máquina virtual se configuró utilizando el modo NAT. Este modo permite que la máquina virtual tenga acceso a Internet mientras permanece aislada de la red local.

Se configuró reenvío de puertos (port forwarding) en VirtualBox para los servicios internos que requieren acceso desde el equipo host, incluyendo:

- CasaOS → puerto de administración web
- Crafty Controller → puerto de panel de servidor
- Servidor Minecraft → puerto estándar de juego

Esto permite interactuar con todos los servicios desde el PC real sin necesidad de exponer la máquina virtual directamente a la red local o a Internet.

Para el acceso externo de jugadores desde Internet se utiliza Playit.gg, que establece un túnel saliente y expone el servidor de forma segura, evitando abrir puertos en el router.

Reglas de reenvío de puertos

Nombre	Protocolo	IP anfitrión	Puerto anfitrión	IP invitado	Puerto invitado
CasaOS	TCP	0.0.0.0	8080	[redacted]	80
Crafty	TCP	0.0.0.0	8111	[redacted]	8111
Minecraft	TCP	0.0.0.0	25565	[redacted]	25565

Este enfoque combina seguridad (NAT + aislamiento) y accesibilidad (puertos hacia host y túnel Playit para Internet), facilitando la administración y el juego remoto.

5. Configuración de CasaOS

5.1. Instalación

La instalación de **CasaOS** se realizó sobre Debian 12 utilizando el script oficial de instalación provisto por el proyecto. Este método permite desplegar CasaOS de forma rápida y confiable, creando todos los servicios y dependencias necesarias automáticamente.

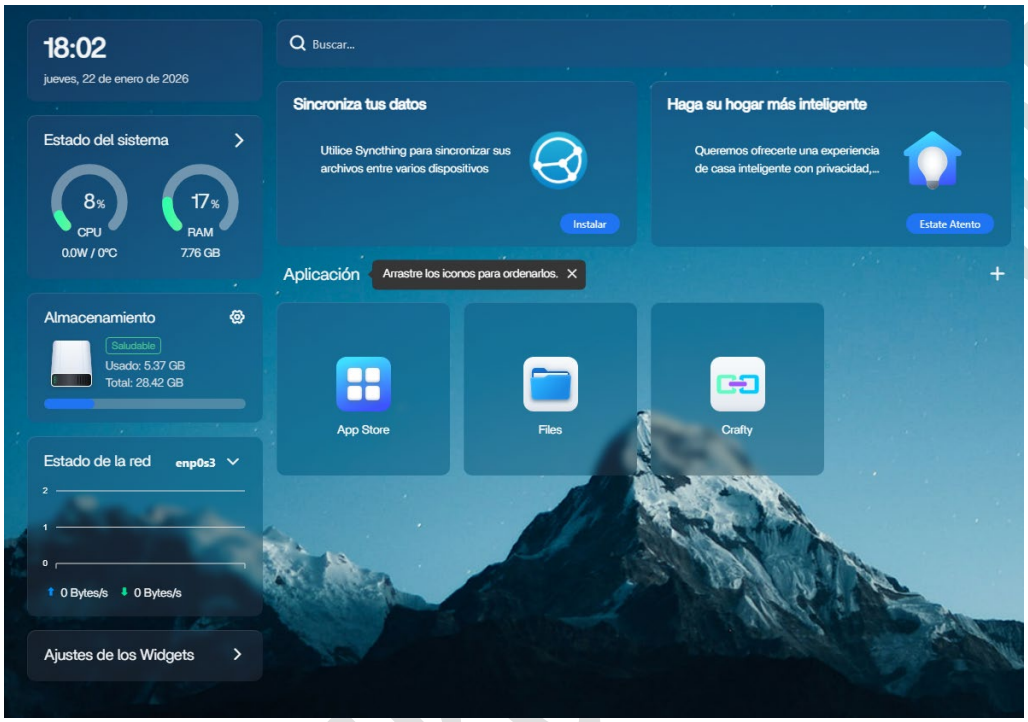
```
--- Made by IceWhale with YOU ---

[  OK  ] Your hardware architecture is : x86_64
[  OK  ] Your System is : Linux
[  OK  ] Your Linux Distribution is : debian
[  OK  ] Memory capacity check passed.
[  OK  ] Disk capacity check passed.
[ INFO ] Updating package manager...
[  OK  ] Update package manager complete.
[ INFO ] Install the necessary dependencies: smartmontools
Selecting previously unselected package smartmontools.
(Reading database ... 33986 files and directories currently installed.)
Preparing to unpack .../smartmontools_7.3-1+b1_amd64.deb ...
Unpacking smartmontools (7.3-1+b1) ...
Setting up smartmontools (7.3-1+b1) ...
```

CasaOS se configura como un servicio del sistema, garantizando que se inicie automáticamente con la máquina virtual.

5.2. Acceso desde el host

Para acceder a CasaOS desde el equipo anfitrión (el PC que ejecuta la máquina virtual), se utiliza *http://localhost:8080* (u otro puerto definido en el reenvío de puertos de VirtualBox). Esto permite gestionar toda la plataforma de manera gráfica sin necesidad de abrir sesión directamente en la VM.



5.3. Servicios gestionados

CasaOS permite instalar y gestionar aplicaciones y servicios de forma centralizada. En este proyecto, se gestionan principalmente:

- **Crafty Controller** → panel de administración del servidor Minecraft
- **Servidor Minecraft Forge** → gestionado indirectamente desde Crafty
- **Tareas auxiliares** → monitoreo de recursos, backups y logs

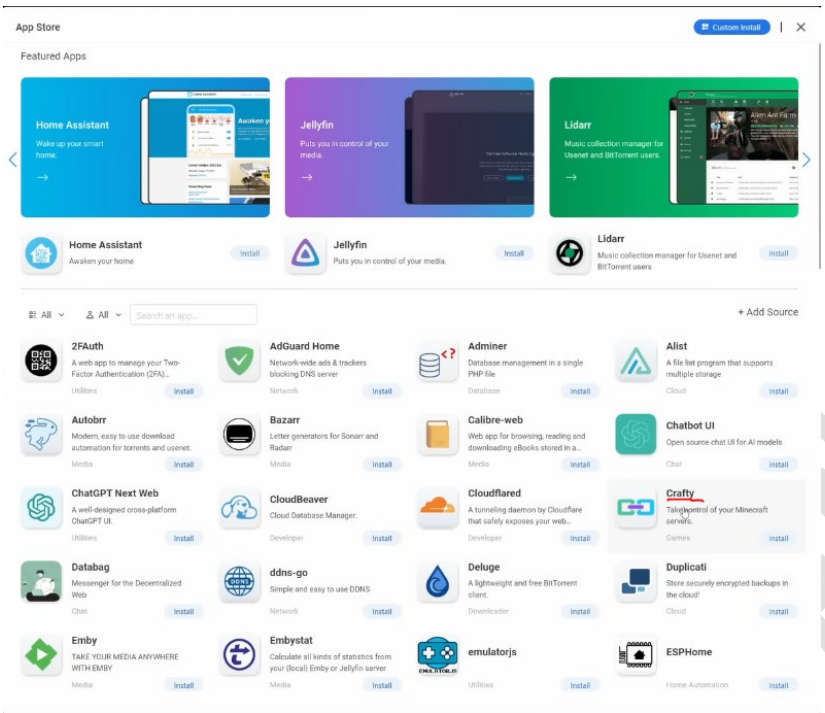
Todos los servicios quedan registrados en CasaOS y pueden configurarse para iniciar automáticamente con la máquina virtual, garantizando disponibilidad constante.

6. Gestión del servidor con Crafty

6.1. Instalación de Crafty

La instalación de **Crafty Controller** se realizó dentro de Debian 12 siguiendo las instrucciones oficiales del proyecto. Se descargó la versión compatible con el sistema y se configuró como un servicio para que pueda ejecutarse de forma automática al iniciar la máquina virtual.

NELLY PIÑEIRO FERNÁNDEZ

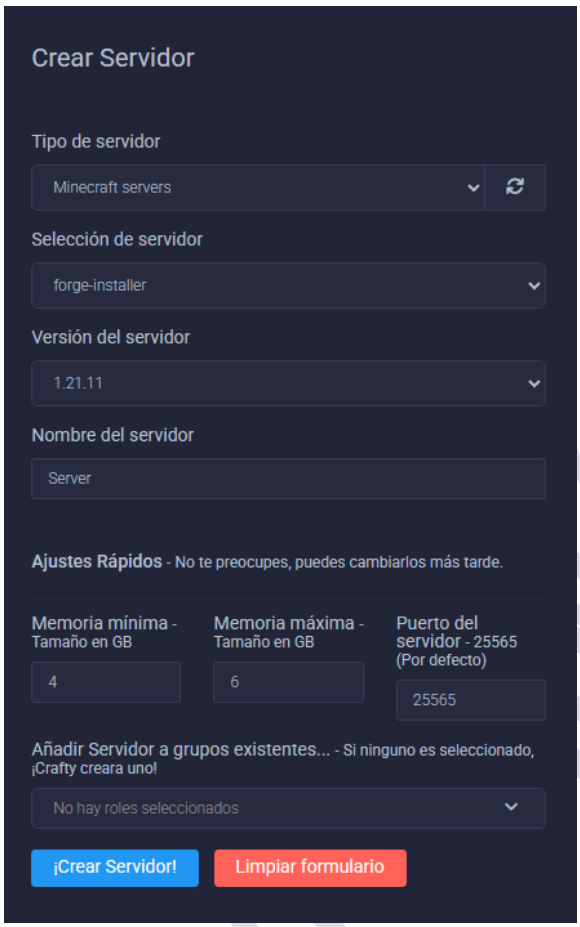


Crafty proporciona un **panel web centralizado** para crear, configurar y monitorizar servidores Minecraft, incluyendo la gestión de mods, memoria, backups y reinicios automáticos.

6.2. Creación del servidor Forge

Desde Crafty se creó un servidor **Minecraft Forge**, asignando la cantidad de memoria RAM recomendada (4–6 GB) y seleccionando la versión de Forge correspondiente a los mods que se van a utilizar.

NELLY PIÑEIRO FERNÁNDEZ



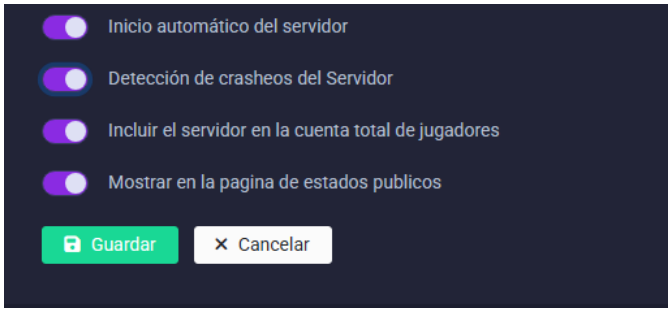
Se configuró el **mundo inicial**, los archivos `server.properties` y `user_jvm_args.txt`, asegurando compatibilidad con los mods y permitiendo ajustes de rendimiento según las necesidades del proyecto.

6.3. Gestión de memoria y rendimiento

Crafty permite definir la **cantidad mínima y máxima de RAM** que el servidor puede utilizar, así como otras opciones de rendimiento de la JVM. Esto garantiza estabilidad durante sesiones de juego y evita que la máquina virtual quede sin recursos.

6.4. Inicio automático y detección de crasheos

El servidor se configuró para **arrancar automáticamente** junto con la máquina virtual y para **reiniciarse en caso de fallo**, asegurando disponibilidad constante. Crafty registra todos los eventos de inicio y errores, permitiendo su monitorización desde la interfaz web.



NELLY PIÑEIRO FERNANDEZ

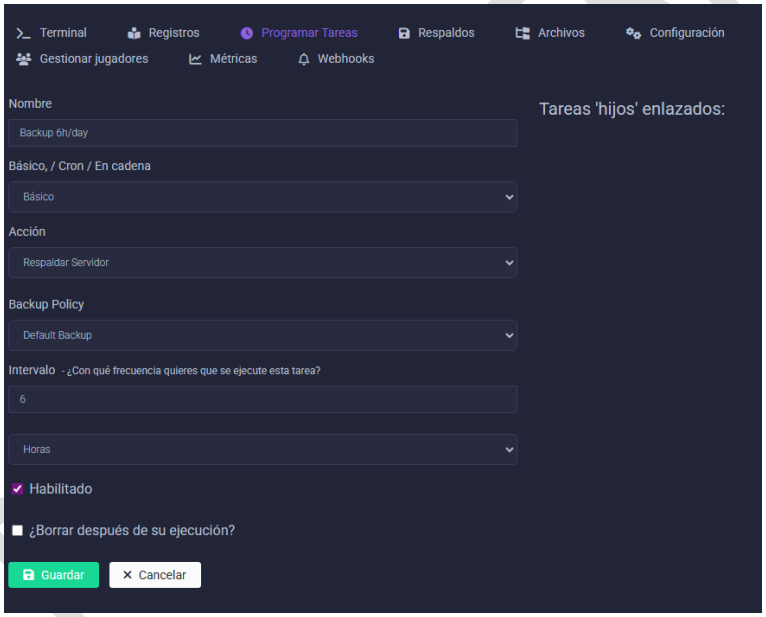
6.5. Backups automáticos

Se configuró en Crafty un sistema de **copias de seguridad automáticas** del servidor Minecraft, con **4 backups diarios** programados cada **6 horas**.

Estos backups incluyen:

- Mundo principal y dimensiones adicionales (`DIM-1`, `DIM1`)
- Archivos de configuración (`server.properties`, `user_jvm_args.txt`)
- Datos de jugadores y entidades
- Carpeta de mods y datapacks

Las copias se almacenan en un directorio específico dentro de la máquina virtual, permitiendo restauraciones rápidas en caso de corrupción de datos o errores críticos durante la ejecución del servidor.



Este sistema asegura la **integridad y continuidad del proyecto**, minimizando riesgos de pérdida de información y facilitando pruebas con mods o cambios de configuración sin comprometer el servidor principal.

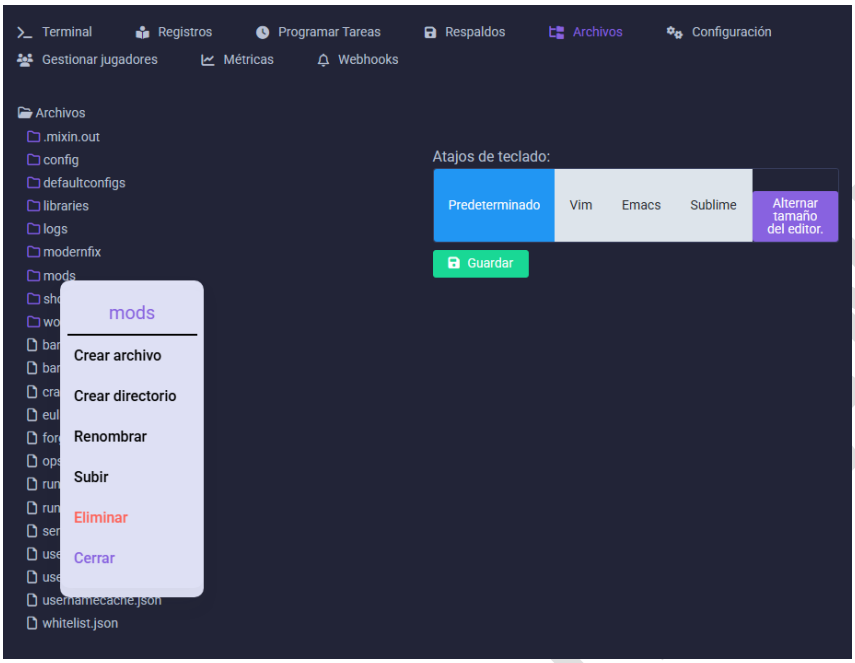
7. Servidor Minecraft Forge con mods

7.1. Preparación del servidor

El servidor se creó en Crafty como un **servidor Forge**, sin necesidad de instalar Forge dentro de la máquina virtual. Crafty gestiona automáticamente la compatibilidad con mods al crear el servidor.

7.2. Gestión de mods (cliente y servidor)

Los **mods del servidor** se agregan subiendo los archivos `.jar` a la carpeta `mods` dentro de Crafty.



Para poder conectarse al servidor, los jugadores deben tener **los mismos mods instalados en su cliente Minecraft** con Forge. En tu caso, Forge está instalado en tu PC real, y los mods en la carpeta `mods` del cliente coinciden con los del servidor.

Crafty permite **reiniciar el servidor** después de añadir o actualizar mods, garantizando que se carguen correctamente y evitando errores de incompatibilidad.

7.3. Uso de Modrinth para gestionar mods en el cliente

Para gestionar los mods en el cliente se utilizó **Modrinth**:

1. Se instaló **Modrinth** en el PC real (cliente).
2. Se creó un **perfil Forge** compatible con la versión del servidor deseada (1.20.1).
3. Se añadieron los **mismos mods que en el servidor** dentro de la carpeta `mods` del perfil de Modrinth.
4. Se verificó que el perfil cargara correctamente los mods antes de conectarse al servidor.

7.4. Generación y reinicio del mundo

Una vez configurado el servidor y añadidos los mods, se generó un mundo nuevo desde Crafty, asegurando que los mods pudieran integrarse correctamente en la generación de terreno, biomas y estructuras.

Si se desea cambiar la semilla o reiniciar el mundo, Crafty permite borrar el mundo existente y generar uno nuevo, manteniendo la misma configuración de mods. Esto asegura que los biomas, estructuras y elementos de los mods se carguen correctamente.

NELLY PIÑEIRO FERNÁNDEZ

Todos los datos del mundo se almacenan dentro de la máquina virtual, incluyendo dimensiones adicionales, entidades, datapacks y configuraciones personalizadas, garantizando que los backups automáticos (configurados en 6.5) capturen toda la información crítica.

7.5. Permisos y administración

Para administrar el servidor se utilizan **comandos de operador (OP)** en Minecraft, que permiten asignar privilegios a jugadores y acceder a comandos administrativos.

Crafty también registra la actividad del servidor, facilitando la monitorización de errores, caídas y reinicios automáticos.

8. Acceso remoto con Playit.gg

8.1. Problemas con Port Forwarding tradicional

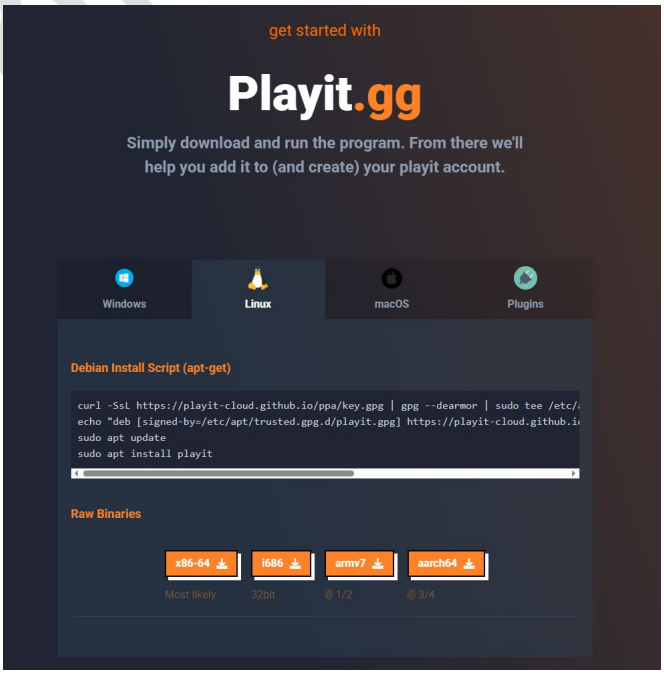
Inicialmente se evaluó la posibilidad de usar **reenvío de puertos en el router** para exponer el servidor Minecraft al exterior. Sin embargo, este método presenta varios inconvenientes:

- Dependencia del modelo de router y configuración manual de puertos.
- Riesgo de exposición directa a Internet, reduciendo la seguridad.
- Dificultad para migrar la máquina virtual a otros equipos sin reconfigurar el router.

Por estas razones, se decidió usar **Playit.gg**, que permite conectividad remota segura mediante túneles salientes.

8.2. Instalación del agente Playit

Se descargó e instaló el **agente Playit** en la máquina virtual para establecer un túnel saliente hacia los servidores de Playit.



NELLY PIÑEIRO FERNÁNDEZ

8.3. Creación del tunel

Mediante el agente, se creó un **túnel TCP** que expone el puerto del servidor Minecraft a un dominio de Playit (`*.joinmc.link`).Esto permite que los jugadores se conecten desde Internet sin necesidad de abrir puertos en el router.

El túnel se validó desde el navegador y la consola del servidor, confirmando que estaba operativo y que el servidor respondía correctamente a conexiones externas.

8.4. Pruebas de conexión externas

Se realizaron pruebas con jugadores externos para verificar la conectividad:

- Los jugadores pudieron conectarse usando el dominio proporcionado por Playit (`*.joinmc.link`).
- El servidor respondió correctamente, cargando mods y mundos según la configuración establecida.

Estas pruebas confirmaron que el acceso remoto funciona de manera segura y confiable, cumpliendo el objetivo de permitir juego externo sin depender de la red local del host.

9. Pruebas y validación del sistema

En esta etapa se verificó que todos los componentes del proyecto funcionaran correctamente de manera conjunta:

1. **Servidor Minecraft Forge con mods**

- Los mundos se cargan correctamente, incluyendo biomas y estructuras de los mods.
- Los jugadores pueden conectarse usando los perfiles de Modrinth con los mismos mods.
- Los permisos de administrador funcionan correctamente y los comandos OP se ejecutan sin errores.

2. **Crafty Controller**

- Reinicios automáticos y detección de crasheos operan como se espera.
- Configuración de memoria y ajustes de rendimiento se aplican correctamente.

3. **CasaOS**

- El panel web es accesible desde el host mediante *http://localhost:8080*.
- Todos los servicios gestionados (Crafty, Minecraft) se muestran correctamente.

4. **Acceso remoto con Playit.gg**

NELLY PIÑEIRO FERNÁNDEZ

- Túnel TCP operativo y jugadores externos pueden conectarse sin necesidad de reenvío de puertos en el router.

5. **Backups automáticos**

- Se generaron correctamente 4 copias diarias, cubriendo mundo, mods, configuración y datos de jugadores.

Esta fase asegura que el proyecto cumpla con estabilidad, seguridad y funcionalidad completa, permitiendo su uso tanto local como remoto sin incidencias.

10. Problemas comunes y soluciones

Durante el desarrollo y prueba del proyecto se identificaron algunos problemas recurrentes y sus soluciones:

1. **Error "server mod list is not compatible"**

- **Causa:** El cliente y el servidor no tenían los mismos mods o versiones incompatibles.

- **Solución:** Asegurarse de que los mods del servidor estén subidos en Crafty y que el cliente tenga exactamente los mismos mods usando Forge o Modrinth.

2. **Fallos de carga de mods individuales (warnings "was not realized")**

- **Causa:** Algunos mods referencian elementos que no existen en la versión de Forge o dependen de otros mods.

- **Solución:** Verificar compatibilidad entre mods y eliminar o actualizar los que causen errores.

3. **Problemas de acceso remoto o túnel Playit**

- **Causa:** El agente Playit no estaba ejecutándose o el túnel estaba inactivo.

- **Solución:** Reiniciar el agente en la VM y verificar que el túnel aparezca activo en la web de Playit.

4. **Errores de memoria o lag en el servidor**

- **Causa:** Asignación insuficiente de RAM o conflictos de JVM.

- **Solución:** Ajustar memoria mínima y máxima en Crafty según la capacidad de la VM y la cantidad de mods (4–6 GB recomendado).

5. **Problemas de permisos en el servidor**

- **Causa:** Jugadores no agregados como OP o falta de privilegios.

- **Solución:** Asignar permisos OP desde Crafty o mediante comandos de consola en el servidor.

NELLY PIÑEIRO FERNÁNDEZ

11. Conclusiones y mejoras futuras

11.1. Conclusiones

1. El proyecto permitió crear un entorno de servidor Minecraft Forge con mods estable y funcional, utilizando máquina virtual, Crafty y CasaOS.

2. La gestión de mods con Crafty y Modrinth garantizó compatibilidad entre servidor y cliente, evitando errores comunes de versiones.

3. El acceso remoto seguro se logró mediante Playit.gg, eliminando la necesidad de exponer puertos del router y simplificando la conexión de jugadores externos.

4. La configuración de backups automáticos y reinicios de servidor asegura continuidad, integridad de datos y facilidad de administración.

11.2. Mejoras futuras

1. Implementar **monitorización avanzada** de recursos de la VM (CPU, RAM, disco) para optimizar rendimiento.

2. Explorar el uso de **plugins o mods de administración** para facilitar reportes y alertas automáticas.

3. Crear **documentación visual completa** con capturas paso a paso de instalación y configuración para facilitar la reproducción del proyecto.

4. Evaluar la **migración a un entorno de servidor dedicado** si se amplía el número de jugadores o la cantidad de mods.

5. Integrar **sistemas de backup externos**, por ejemplo en la nube, para mayor seguridad y facilidad de restauración.

NELLY PIÑEIRO FERNÁNDEZ